

NuclearEulerCloud / NuclearEulerParcel

A two-phase-Euler-aware Lagrangian particle cloud for tom-v2606, built on NuclearCloud/NuclearParcel

Prepared for Tommaso Pernatsch · tom-v2606 (OpenFOAM v2606, myIntermediate fork) · September 2026

Contents

1. Summary
2. The problem and the design goal
3. Architecture: where the new classes sit
4. What "reading both phases" actually means, field by field
5. The physics: heat transfer, dispersion, and the phase-switch force framework
6. Files created
7. Files modified, and why
8. The fvOptions coupling layer
9. Build and validation
10. A real bug found along the way (and one ruled out)
11. Known limitations and natural next steps
12. How to use it in a case
13. Revision: removing vorticity dispersion, a placeholder χ -switch, and a real two-phase test
14. Revision: χ as a boolean stochastic switch, vorticity removed fork-wide, FunctionYPlusDispersionRAS retired
15. Revision: TwoPhaseBrownianMotionForce moved to myTurbulence

1. Summary

You asked for a Cloud/Parcel pair for two-phase Euler–Euler situations, built on `NuclearCloud` / `NuclearParcel`, able to read each phase's turbulence quantities, densities, velocities and temperatures, plus the void fraction. That's `NuclearEulerCloud` and `NuclearEulerParcel`, added to `src/myIntermediate`, plus a new `fvOptions` class (`nuclearEulerCloudSource`) that feeds them from a live `reactingMultiphaseEulerFoam` case with zero solver changes — the same "glue it on via `fvOptions`" approach already used for the single-phase nuclear cloud earlier in this project.

Concretely:

- **NuclearEulerParcel** literally derives from `NuclearParcel` (which derives from `KinematicParcel`), so every existing force, injection, patch-interaction and decay-heat model keeps working unmodified. Every carrier-field quantity it reads is transient, re-interpolated every tracking sub-step, exactly like the primary phase's fields already are — the one genuinely new piece of persisted per-parcel state is the phase-switch parameter χ described next.
- **NuclearEulerCloud** literally derives from `NuclearCloud`, adding ten new carrier-field references: the primary phase's own turbulence k/ϵ (which plain `KinematicCloud` has no concept of at all), and the secondary phase's void fraction, velocity, density, viscosity, temperature and turbulence k/ϵ .
- Convective heat transfer is generalised to a void-fraction-weighted two-phase closure that reduces exactly to plain `NuclearParcel` behaviour as the secondary phase's local volume fraction goes to zero — a checked property, not just a claim (see the validation run below). Real dispersion models still live in `myTurbulence`, not `myIntermediate` — but the cloud now exposes a *second*, independently-selectable dispersion slot, `dispersion2()`, that a parcel's `calcDispersion()` consults only when it is switched ($\chi > 0$) onto the secondary phase *and* that phase's own turbulence model reports itself as RAS; three new `TwoPhase*DispersionRAS` models in `myTurbulence` implement it (§5.2).
- **A new per-parcel switch parameter, χ (chi)**, in $\{0,1\}$ and always starting at 0, selects which phase's properties every *coupled* particle force is evaluated against. Seven new two-phase-aware force models — drag, lift, gravity, virtual mass, pressure gradient, thermophoretic and Brownian motion — each compute $\mathbf{F} = \mathbf{F}_1(1-\chi) + \mathbf{F}_2\chi$, registered as additional selectable force types alongside (not replacing) the existing single-phase ones. No model exists yet to set χ to anything but 0 (deliberately future work), so today every one of these reduces exactly to its familiar primary-phase-only counterpart.
- Everything compiles clean (`wmake libso`, exit 0, zero warnings-as-errors, zero undefined symbols on `ldd -r`) across all three libraries — `myIntermediate`, `myTurbulence`, and `nuclearCloudFvOptions` — and was run end-to-end against a real, unmodified `reactingMultiphaseEulerFoam` tutorial (air/water bubble column) with the new `fvOption`, all seven new forces, and both dispersion slots selected, tracking five parcels.
- Along the way, one real, previously-undiscovered bug was found and fixed in the *existing* single-phase `nuclearCloudSource` `fvOption` (a gravity field lookup that only fails against a real multiphase solver, never exercised before); a separate solver instability was thoroughly bisected and proven to be *unrelated* to any of this new code (§10.2); and a real dictionary-convention gotcha for multi-force selection lists was found and documented (§10.3).

2. The problem and the design goal

`NuclearCloud` / `NuclearParcel` (and everything under them — `KinematicCloud` / `KinematicParcel`) were written assuming there is exactly one continuous carrier phase: one ρ , one U , one μ , one T . That's the right assumption for a solver like `nuclearParcelFoam` or `buoyantBoussinesqPimpleFoam`. It stops being enough the moment the "continuous phase" the parcel is actually tracked through is itself one phase of a genuine two-fluid Euler–Euler system — a bubble column, say, where a dense particle is carried by the liquid but is also physically sitting in a mixture whose local gas holdup, gas temperature and gas-phase turbulence are all real, available, and arguably relevant to how that particle behaves.

The brief was specific about what "aware of the second phase" should mean: the parcel needs to be able to *read* the second phase's density, velocity, temperature and turbulence quantities, and the local void fraction — the same way it already reads those things for its one carrier phase today.

Two design choices followed from reading the brief literally:

1. **"Based on NuclearCloud/Parcel"** was taken to mean actual C++ inheritance, not "inspired by".
`NuclearEulerCloud<CloudType>` derives from `NuclearCloud<CloudType>` ;
`NuclearEulerParcel<ParcelType>` derives from `NuclearParcel<ParcelType>`. Every existing model — drag, gravity, heat transfer, decay heat, radiation, patch interaction, injection — is inherited unchanged and keeps working with zero modification to any existing file in the drag/injection/heat-transfer submodel libraries.
2. **Momentum and other coupled forces are genuinely two-phase, gated by a switch, not a voidage correlation.** Rather than building one fixed voidage-blended drag/lift/... correlation, each coupled force is computed *twice* — once with each phase's own fields — and blended by a new per-parcel parameter χ (chi) that will eventually be set by a dedicated "which phase am I actually in" switching model (e.g. detecting particle-bubble attachment). That model doesn't exist yet, so χ is always 0 today (see §5.3), but the force machinery is already fully two-phase-aware and ready for it.

None of this touches the *existing*, shared `ParticleForce` submodels library that every other parcel type in the fork (Reacting, Colliding, ...) also depends on: the seven new two-phase force classes are *additional* selectable types, registered only against `basicNuclearEulerCloud`, living in their own new `submodels/NuclearEuler/ParticleForces` directory. Every existing single-phase force (`sphereDrag`, `gravity`, ...) is completely unmodified and still selectable as before.

3. Architecture: where the new classes sit

OpenFOAM's Lagrangian intermediate library builds each concrete parcel/cloud pair by nesting class templates, each layer adding one family of physics on top of the last. `NuclearCloud` already sits on top of `KinematicCloud` this way; `NuclearEulerCloud` simply adds one more layer on top of that:

```
basicNuclearEulerParcel =  
  NuclearEulerParcel< NuclearParcel< KinematicParcel< particle > > >
```

```
basicNuclearEulerCloud =
    NuclearEulerCloud< NuclearCloud< KinematicCloud< Cloud< basicNuclearEulerParcel >
> > >
```

Compare with the existing `basicNuclearParcel = NuclearParcel<KinematicParcel<particle>> / basicNuclearCloud = NuclearCloud<KinematicCloud<Cloud<basicNuclearParcel>>>` — the new types are exactly one layer further out, nothing underneath changed.

Two non-templated marker/base classes complete the pattern, mirroring how `nuclearCloud.H` already exists purely so non-templated code can refer to "a NuclearCloud" by interface rather than by its full template instantiation:

File	Role
<code>clouds/baseClasses/ nuclearEulerCloud/ nuclearEulerCloud.H/.C</code>	Minimal RTTI marker class (<code>TypeName("nuclearEulerCloud")</code>), plus the small <code>eulerCarrierPhaseFields</code> helper described in §4.

Why one more layer, rather than folding this into NuclearCloud itself

Every existing consumer of `NuclearCloud / NuclearParcel` (the single-phase `nuclearParcelFoam / msfrParcelFoam` solvers, the existing `nuclearCloudSource` fvOption) keeps compiling and behaving identically, because nothing about `NuclearCloud / NuclearParcel` themselves changed. The two-phase awareness is opt-in: you only get it, and only pay for the extra ten interpolators being constructed and refreshed every sub-step, if you explicitly ask for `basicNuclearEulerCloud` instead of `basicNuclearCloud`.

A subtlety that shaped the Cloud-side implementation

Every layer in this template stack — `KinematicCloud`, `NuclearCloud`, and now `NuclearEulerCloud` — defines its own `evolve() / preEvolve()`, even though each one's body looks like a trivial pass-through. This isn't boilerplate for its own sake: inside `NuclearCloud<CloudType>::evolve()`, the expression `*this` has the *static* type `NuclearCloud<CloudType>&`, and that's the type the compiler uses to instantiate the rest of the per-particle tracking chain (`trackingData` construction, every parcel's `calc()`). If `NuclearEulerCloud` didn't define its own `evolve()`, name lookup would find and call `NuclearCloud`'s version instead — and the whole tracking chain would then be instantiated against a cloud type that doesn't have `alpha1() / U2() / ...` at all, a compile error at best and silently-wrong behaviour at worst if any of it happened to still typecheck. So `NuclearEulerCloud` re-declares `evolve() / preEvolve()` itself, exactly mirroring what `NuclearCloud` and `ReactingCloud` already do for the identical reason. `postEvolve()` was added to that list for the same reason once the second dispersion slot needed its own `cacheFields(true) / cacheFields(false)` bracket around every `evolve()` call (§5.2) — `KinematicCloud` is the only layer below that defined it before, and nothing above it needed to hook into post-evolve cleanup until now.

The other place this matters is `clone() / cloneBare()`, which *are* virtual in this hierarchy. `NuclearEulerCloud` overrides both (returning a full `NuclearEulerCloud`, not a base-sliced `NuclearCloud`) so that steady-state cloud copying (`storeState() / restoreState()`), inherited unchanged from `NuclearCloud`) keeps constructing the right concrete type even though it's `NuclearCloud`'s own code doing the calling.

4. What "reading both phases" actually means, field by field

The primary phase keeps using exactly the fields `NuclearCloud` already stores (`rho`, `U`, `mu`, `vort`, `T`). `NuclearEulerCloud` adds:

Field	Accessor	Source
Primary phase volume fraction	<code>alpha1()</code>	New — plain <code>KinematicCloud</code> has no void-fraction concept at all
Primary phase turbulence k , ϵ	<code>k1()</code> , <code>epsilon1()</code>	New — used by the two-phase Brownian motion force (§5.3)
Secondary phase volume fraction	<code>alpha2()</code>	New
Secondary phase velocity	<code>U2()</code>	New
Secondary phase density	<code>rho2()</code>	New
Secondary phase viscosity	<code>mu2()</code>	New
Secondary phase temperature	<code>T2()</code>	New
Secondary phase turbulence k , ϵ	<code>k2()</code> , <code>epsilon2()</code>	New
Secondary phase vorticity	<code>vort2()</code>	New — feeds the two-phase vorticity dispersion model (§5.2)
Is the primary/ secondary phase's own turbulence model RAS?	<code>k1IsRAS()</code> , <code>k2IsRAS()</code>	New — read once from each phase's <code>turbulenceProperties.<phase> simulationType</code> entry by <code>nuclearEulerCloudSource</code> ; gates the secondary-phase dispersion dispatch (§5.2)
Secondary-phase dispersion model	<code>dispersion2()</code>	New — a second, independently-selectable <code>DispersionModel</code> slot (§5.2)

These are seven fields belonging to the secondary phase plus three extra belonging to the primary phase — ten new `const volScalarField&/const volVectorField&` references in total. A flat 16-argument constructor was avoided by bundling the seven secondary-phase fields into one small aggregate, `eulerCarrierPhaseFields`, declared alongside the marker base class:

```
class eulerCarrierPhaseFields
{
public:
    const volScalarField& alpha;
    const volVectorField& U;
    const volScalarField& rho;
    const volScalarField& mu;
    const volScalarField& T;
    const volScalarField& k;
    const volScalarField& epsilon;
    ...
};
```

so the cloud constructor reads as "the usual NuclearCloud arguments, plus `alpha1 / k1 / epsilon1` for the primary phase, plus one `phase2` bundle" — nine logical pieces of information instead of sixteen positional scalars-and-fields that are easy to transpose by accident.

Parcel side: trackingData, not new persisted state

This is the detail that keeps the whole design lightweight. `KinematicParcel` already never stores the carrier's `rho / U / mu / vort` *on the particle itself* — those are interpolated fresh to the particle's position every tracking sub-step and cached in a transient `trackingData` object that lives only for the duration of one `evolve()` call. `NuclearEulerParcel::trackingData` does exactly the same thing for all ten new fields: ten more `interpolation<Type>` objects, built once per `evolve()` from `cloud.solution().interpolationSchemes()` (so the case dict controls the interpolation scheme per field, same as every other carrier field), and ten cached scalars/vector refreshed in `setCellValues()` every sub-step:

```
td.alpha1c() = clamp(td.alpha1Interp().interpolate(coords, tetIs), 0, 1);
td.alpha2c() = clamp(td.alpha2Interp().interpolate(coords, tetIs), 0, 1);
td.U2c()      = td.U2Interp().interpolate(coords, tetIs);
td.T2c()      = td.T2Interp().interpolate(coords, tetIs); // floored at TMin, like Tc()
td.k1c()      = max(td.k1Interp().interpolate(coords, tetIs), 0);
... (epsilon1c, k2c, epsilon2c likewise floored at zero)
```

Consequently every extra *field* quantity this class reads costs zero new persisted per-parcel state. There is exactly one genuinely new piece of per-parcel state, added deliberately: the phase-switch parameter `chi_` (§5.3), because unlike the transient carrier fields it must actually persist between time steps (and, in the future, be written to by a switching model) rather than being recomputed from scratch every sub-step.

`NuclearEulerParcelIO.C` reads/writes exactly that one new field — a single `IOfield<scalar>` named `"chi"` — and otherwise passes everything through to `NuclearParcel`'s own I/O unchanged. The other new per-instance state is one constant, dictionary-supplied model parameter (`alphaSmall`, §5.1) added to `constantProperties`, which needs no I/O of its own since constant properties are read once from the case dictionary, not written per time step.

5. The physics: two closures that use the second phase

Reading the extra fields is only useful if something actually does something with them. Heat transfer (§5.1) and the new two-phase force framework (§5.3) are where that happens; dispersion (§5.2) deliberately stays untouched. Every closure that was added is designed to **reduce exactly to plain NuclearParcel behaviour** as the secondary phase's local volume fraction (or the switch parameter χ) goes to zero — a genuine correctness property, not just a convenient default, and the one directly checked by the validation run in §9.

5.1 Void-fraction-weighted two-phase convective heat transfer

Plain `NuclearParcel::calcHeatTransfer()` computes one heat-transfer coefficient $htc = \text{Nu}(\text{Re}, \text{Pr}) \cdot \kappa / d$ from the slip Reynolds number against the single carrier, and integrates

$$dT/dt \cdot (mC_p) = htc \cdot A_s \cdot (T_c - T) + (\text{radiation}) + (\text{decay heat})$$

`NuclearEulerParcel::calcHeatTransferTwoPhase()` (in `NuclearEulerParcel.C`) generalises this to a two-resistance-in-parallel closure: the *same* $\text{Nu}(\text{Re}, \text{Pr})$ correlation is evaluated twice — once with the primary phase's own slip Reynolds number Re_1 (against U_c), once with the secondary phase's Re_2 (against U_{2c}) — and the two resulting coefficients are combined into an effective coefficient and an htc -weighted mean carrier temperature:

$$\begin{aligned} w_1 &= \alpha_{1c} / (\alpha_{1c} + \alpha_{2c} + \epsilon), \quad w_2 = \alpha_{2c} / (\alpha_{1c} + \alpha_{2c} + \epsilon) \\ htc_{\text{eff}} &= w_1 \cdot htc_1 + w_2 \cdot htc_2 \\ T_{c,\text{eff}} &= (w_1 \cdot htc_1 \cdot T_{1c} + w_2 \cdot htc_2 \cdot T_{2c}) / htc_{\text{eff}} \end{aligned}$$

and $htc_{\text{eff}}/T_{c,\text{eff}}$ are substituted for htc/T_c in exactly the same integration formula `NuclearParcel` already uses (same `cloud.TIntegrator().delta(...)` call, same radiation and decay-heat terms, same `TMin / TMax` clamping). Two numerical details:

- ϵ above is `constantProperties::alphaSmall()` — a new, optional (default 10^{-6}) dictionary entry that floors the weighting denominator against the degenerate case of both interpolated void fractions collapsing toward zero at the same point (e.g. right at a wall or free surface where both fields are being blended by their own numerical schemes). It is *not* a physical parameter; it exists purely so a division by a near-zero sum can never produce a NaN or an unbounded weight.
- The formula is written so that when $w_2 \rightarrow 0$ (secondary phase locally absent), $htc_{\text{eff}} \rightarrow htc_1$ and $T_{c,\text{eff}} \rightarrow T_{1c}$ exactly — i.e. it degenerates to precisely the single-phase `NuclearParcel` formula, not an approximation of it. This is the property the validation run in §9 actually exercises and confirms.

5.2 Turbulent dispersion: a second, RAS-gated slot for the secondary phase

An earlier iteration of this class added an ad-hoc void-fraction-weighted stochastic dispersion correction directly inside `NuclearEulerParcel`. That was removed on explicit instruction: the real dispersion models — `DispersionRASModel` and its derivatives (`GradientDispersionRAS`, `StochasticDispersionRAS`,

`VorticityDispersionRAS`, ...) — live in a separate library, `myTurbulence`, not in `myIntermediate` where this class lives, and dispersion physics has no business being bolted onto the `parcel` class directly.

That still leaves a real question: a two-phase cloud can have either phase turbulent, both, or neither, and a parcel dispersing under the primary phase's turbulence should *also* pick up extra agitation from the secondary phase wherever it is locally significant (e.g. bubble-induced turbulence acting on a particle nominally carried by the liquid) — but only where that makes physical sense. The design settled on:

- **The primary phase's dispersion is completely untouched.** `NuclearEulerCloud`'s inherited `dispersion()` slot, and everything `ParcelType::calcDispersion()` does with it, is exactly what a plain `NuclearParcel` already does — zero change to any shared `DispersionModel` / `KinematicParcel` code.
- **A second, independent slot, `dispersion2()`, is added purely on the `NuclearEulerCloud` / `NuclearEulerParcel` layer.** It is selected by its own dictionary entry, `dispersionModel2`, read via a dict-key-rename trick (`DispersionModel::New()` hardcodes the key `"dispersionModel"`, so `NuclearEulerCloud`'s constructor builds a throwaway copy of `subModelProperties()` with `dispersionModel2` renamed to `dispersionModel` in the copy before handing it to `New()`; if the entry is absent it defaults to `none`, so an existing case with no `dispersionModel2` entry behaves exactly as before).
- **`NuclearEulerParcel::calcDispersion()` is reinstated, but purely as a dispatcher**, not a new physics model: it always calls `ParcelType::calcDispersion()` unchanged (primary phase), then additionally calls `ccloud.dispersion2().update(...)` against the secondary phase's own fields (`td.U2c()` / `td.rho2c()` / `td.mu2c()` / `td.vort2c()`) **only if both of the following hold**:
 1. $\chi > 0$ — the parcel is actually switched (even partially) onto the secondary phase; and
 2. `ccloud.k2IsRAS()` — the secondary phase's own turbulence model reports itself as RAS, not laminar or LES.

if ($\chi > 0$ && `k2IsRAS`): $\mathbf{U}_{2c} \leftarrow \text{dispersion2().update(dt, ..., } \mathbf{U}_{2c}, \rho_{2c}, \mu_{2c}, \mathbf{vort}_{2c}, \dots)$

The RAS check exists because an LES phase's resolved-vs-modelled turbulence split gives its k a different physical meaning than a RAS k — it is not a valid input to the RAS eddy-lifetime correlations these dispersion models use, and silently feeding it in would produce a number, not a meaningful one. `k1IsRAS()` / `k2IsRAS()` are computed once per `correct()` call by `nuclearEulerCloudSource`, directly from each phase's own `turbulenceProperties.<phase>` dictionary: `turbulenceModel` derives from `I0dictionary`, so the already-looked-up `phaseCompressibleTurbulenceModel&` can be read as a plain dictionary and its `simulationType` entry (`"laminar"` / `"RAS"` / `"LES"`) compared directly — no separate lookup or new field needed.

Where the $k2/\epsilon2$ fields actually come from, and why it's simpler than the primary phase's own machinery

`DispersionRASModel::kModel()` / `epsilonModel()` (the primary-phase base class) has to locate the turbulence model in the mesh object registry and return its k/ϵ as a `tmp<>`, with a full own/don't-own bookkeeping dance, because plain `KinematicCloud` has no turbulence awareness of its own to hand it a persistent field instead. `NuclearEulerCloud` has already solved that problem for a different reason (§4): it stores `k2()` / `epsilon2()` as genuine persistent fields, refreshed once per PIMPLE iteration by

`nuclearEulerCloudSource`. So the new base class, `TwoPhaseDispersionRASModel`, just returns references to those directly — no `tmp<>`, no ownership flags, no registry lookup. The one piece of unavoidable bookkeeping it does still need is the same static-owner-type gotcha documented throughout this report: `DispersionModel<CloudType>::owner()` is fixed by the `makeDispersionModelType()` macro to `CloudType::kinematicCloudType` — the bare `KinematicCloud` layer, which has no `k2()` / `epsilon2()` of its own — so an explicit `static_cast<const basicNuclearEulerCloud&>(this->owner())` is required to reach them, exactly the pattern the two-phase `ParticleForces` (§5.3) already use for the identical reason.

Model / selectable type name	What it does
<code>TwoPhaseGradientDispersionRAS</code> <code>twoPhaseGradientDispersionRAS</code>	Direct port of <code>GradientDispersionRAS</code> : perturbs the secondary-phase slip velocity in the direction of $-\nabla k_2$, Gaussian-distributed, eddy lifetime $\min(k_2/\epsilon_2, c_{ps} k_2^{1.5}/\epsilon_2/ U_{rel})$
<code>TwoPhaseStochasticDispersionRAS</code> <code>twoPhaseStochasticDispersionRAS</code>	Direct port of <code>StochasticDispersionRAS</code> : isotropic random perturbation, variance set by k_2/ϵ_2 , same eddy-lifetime formula
<code>TwoPhaseVorticityDispersionRAS</code> <code>TwoPhaseVorticityDispersionRAS</code>	Direct port of <code>VorticityDispersionRAS</code> : perturbation direction biased by the componentwise product of vort _{2c} and U_{2c} , same eddy-lifetime formula

`FunctionYPlusDispersionRAS` (wall-distance-dependent) was not ported — see §11. All three ported models are registered via a new macro, `makeNuclearEulerParcelTurbulenceDispersionModels(CloudType)`, called only for `basicNuclearEulerCloud`, into the *same* runtime-selection table the primary-phase models already share (both are keyed by `CloudType::kinematicCloudType`, which is identical for the `dispersionModel` and `dispersionModel2` entries of the same cloud) — so `dispersionModel2` can select any of `none` / `twoPhaseGradientDispersionRAS` / `twoPhaseStochasticDispersionRAS` / `TwoPhaseVorticityDispersionRAS`, and (harmlessly, if oddly) even one of the primary-phase-only model names, since they all share one table.

While χ is always 0 today (§5.3), `cloud.k2IsRAS()` being false for every currently-existing tutorial case (both phases start laminar) means the new dispatch branch is provably dormant at runtime *twice over* right now — but it is fully built, wired into `preEvolve()` / `postEvolve()`'s `cacheFields(true)` / `cacheFields(false)` bracket exactly like the primary dispersion model, and the validation run in §9 confirms `dispersionModel2` `twoPhaseGradientDispersionRAS`; selects and constructs correctly end-to-end with zero effect on the existing, already-validated results.

5.3 The phase-switch parameter χ and the two-phase force framework

Every coupled particle force (the ones affecting the parcel's own momentum: drag, lift, gravity, virtual mass, pressure gradient, thermophoresis, Brownian motion) can now be evaluated against *either or both* Euler phases, selected by a new per-parcel state variable:

$$\mathbf{F} = \mathbf{F}_1(1-\chi) + \mathbf{F}_2\chi$$

where $\mathbf{F}_1/\mathbf{F}_2$ are the *same* force correlation evaluated once with the primary phase's fields and once with the secondary phase's, and $\chi \in [0,1]$ is `NuclearEulerParcel::chi_` — a new, genuinely persisted per-parcel field (unlike everything else this class reads, which stays transient in `trackingData`; see §4). χ always starts at 0 and nothing in this codebase currently changes it: there is no "switching model" yet to decide, physically, when a particle should be treated as belonging to the secondary phase instead of the primary one (the intended future use is something like detecting particle–bubble attachment). Until that model exists, every one of these seven forces reduces *exactly* to its familiar primary-phase-only value, since $(1-0)=1$ and 0-weighted $\mathbf{F}_2$ contributes nothing — this is checked by the validation run in §9, which selects all seven and observes them behave identically to their plain single-phase counterparts.

Where the code lives, and why it's separate from the shared submodels library

All seven new classes live in a new directory, `src/myIntermediate/submodels/NuclearEuler/ParticleForces/`, registered via a new macro, `makeNuclearEulerParcelForces(CloudType)` (in `parcels/include/makeNuclearEulerParcelForces.H`), called only for `basicNuclearEulerCloud` — alongside, not instead of, the existing `makeNuclearParcelForces(CloudType)` call that registers every plain single-phase force. Both sets of forces are simultaneously selectable in the same case dictionary. None of the existing shared force classes (`GravityForce`, `SphereDragForce`, ..., used by every other parcel type in the fork) needed to change, since each two-phase force is a new, independent class, not a modification of an existing one. Each new class only compiles when actually instantiated against a parcel type whose `trackingData` exposes the secondary-phase accessors (i.e. `basicNuclearEulerParcel` and its derivatives) - this is enforced implicitly by C++ template instantiation rules, not by an explicit constraint.

Force	Class / selectable type name	$\mathbf{F}_1/\mathbf{F}_2$ formula
Gravity (buoyancy-corrected)	<code>TwoPhaseGravityForce</code> <code>twoPhaseGravity</code>	$m \cdot \mathbf{g}(1 - \rho_{ic}/\rho_p)$
Sphere drag	<code>TwoPhaseSphereDragForce</code> <code>twoPhaseSphereDrag</code>	$Sp_i = \frac{3}{4} \cdot m \cdot \mu_0 \cdot C_D Re_i / (\rho_p d^2)$ (Putnam/AOB correlation, same as plain <code>sphereDrag</code>)
Lift	<code>TwoPhaseLiftForce (base) /</code> <code>TwoPhaseSaffmanMeiLiftForce</code> <code>twoPhaseSaffmanMeiLift</code>	$(m/\rho_p) \cdot \rho_{ic} \cdot C_{l,i} \cdot [(\mathbf{U}_{ic} - \mathbf{U}_p) \times (\nabla \times \mathbf{U}_{ic})]$
Pressure gradient	<code>TwoPhasePressureGradientForce</code> <code>twoPhasePressureGradient</code>	$m \cdot (\rho_{ic}/\rho_p) \cdot D\mathbf{U}_{ic}/Dt$
Virtual (added) mass	<code>TwoPhaseVirtualMassForce</code> <code>twoPhaseVirtualMass</code>	$C_{vm} \cdot$ (pressure-gradient formula above); $massAdd = m \cdot (\rho_{ic}/\rho_p) \cdot C_{vm}$
Thermophoretic	<code>TwoPhaseThermophoreticForce</code> <code>twoPhaseThermophoretic</code>	$-D_{t,i} \cdot \nabla T_i / T_{ic}$ (own $K_{c,i}/\lambda_i$ per phase - see below)
Brownian motion	<code>TwoPhaseBrownianMotionForce</code> <code>twoPhaseBrownianMotion</code>	Li & Ahmadi (1992) correlation, independently per phase, each with its own random spherical kick

Each is a direct two-phase generalisation of the plain, single-phase force it is named after (`GravityForce`, `SphereDragForce`, `SaffmanMeiLiftForce`, `PressureGradientForce` / `VirtualMassForce`,

`ThermophoreticForce`, and `myTurbulence`'s `BrownianMotionForce` respectively) — same correlation, evaluated twice. A few implementation details worth being explicit about:

- **`Re2` reuses the same particle-material convention as `Re1`.** Drag and lift need a slip Reynolds number against each phase. `NuclearParcel`'s established (if slightly unusual) convention evaluates the primary phase's Re/μ from the *particle's own* ρ_0/μ_0 , not the true carrier density/viscosity (see §5.4). Re_2 is computed the same way — same ρ_0/μ_0 , only the slip velocity changes (against U_{2c} instead of U_c) — so the two remain on the same, consistent footing rather than mixing conventions.
- **Pressure-gradient/virtual-mass and lift each cache a second mesh field.** The plain `PressureGradientForce` / `LiftForce` compute DU/Dt or $\text{curl}(\mathbf{U})$ once, from one named field ("U", default). The two-phase versions do this twice, from two dict-configurable field names (`U1` / `U2`, defaulting to `"U.water"` / `"U.air"`) — both are genuine, already-registered solver fields (unlike $\rho/\mu/\alpha$, these need no persistent-copy workaround), so this needed no new `fvOption` plumbing.
- **Thermophoretic force takes separate carrier properties per phase** (`Kc1` / `Kc2` thermal conductivity, `lambda1` / `lambda2` molecular mean free path) since a gas and a liquid carrier can differ by orders of magnitude in exactly these properties; the particle's own thermal conductivity `Kp` is shared, since it doesn't depend on which phase is being considered.
- **Brownian motion reuses `trackingData`'s already-interpolated turbulence fields** (`td.k1c()` / `td.k2c()`) rather than doing its own mesh lookup the way `myTurbulence`'s single-phase `BrownianMotionForce` does — those fields already exist on `NuclearEulerParcel::trackingData` for exactly this kind of use (§4).

5.4 What deliberately did not change

Only one drag correlation (sphere drag) and one lift correlation (Saffman-Mei) were given two-phase siblings; the fork's other variants (`WenYuDrag`, `ErgunWenYuDrag`, `PlessisMasliyahDrag`, `NonSphereDrag`, `TomiyamaLift`) were not, though each would follow the identical pattern (see §11). Reynolds numbers for both phases reuse the exact same (slightly unusual) convention `NuclearParcel` already has: `Re()` is evaluated using the *particle's own material* ρ_0/μ_0 in the "carrier-side" argument slots (via `calcSurfaceValues()`, unchanged, inherited), not the true carrier density/viscosity. That's a pre-existing characteristic of this incompressible-parcel variant, not something introduced or "fixed" here.

6. Files created

myIntermediate library (src/myIntermediate/)

File	What it is
clouds/baseClasses/nuclearEulerCloud/nuclearEulerCloud.H	Marker base class + <code>eulerCarrierPhaseFields</code> aggregate (§3, §4)
clouds/baseClasses/nuclearEulerCloud/nuclearEulerCloud.C	<code>defineTypeNameAndDebug</code> for the marker class
clouds/Templates/NuclearEulerCloud/NuclearEulerCloud.H	Templated cloud class declaration
clouds/Templates/NuclearEulerCloud/NuclearEulerCloudI.H	Trivial inline accessors (<code>alpha1()</code> , <code>U2()</code> , ...)
clouds/Templates/NuclearEulerCloud/NuclearEulerCloud.C	Constructors, <code>evolve()</code> / <code>preEvolve()</code> , <code>info()</code> (§3)
clouds/derived/basicNuclearEulerCloud/basicNuclearEulerCloud.H	The concrete typedef
parcels/Templates/NuclearEulerParcel/NuclearEulerParcel.H	Templated parcel class declaration: <code>constantProperties</code> , <code>trackingData</code> , the new <code>chi_</code> field and its accessors, method signatures
parcels/Templates/NuclearEulerParcel/NuclearEulerParcelI.H	<code>constantProperties</code> implementation, trivial constructors (each initialising <code>chi_(0.0)</code>), <code>chi()</code> accessors (§4, §5.3)
parcels/Templates/NuclearEulerParcel/NuclearEulerParcelTrackingDataI.H	<code>trackingData</code> constructor (builds the ten interpolators) and accessors
parcels/Templates/NuclearEulerParcel/NuclearEulerParcel.C	<code>setCellValues()</code> , <code>calc()</code> , <code>calcHeatTransferTwoPhase()</code> (§5.1) — no <code>calcDispersion()</code> override (§5.2)
parcels/Templates/NuclearEulerParcel/NuclearEulerParcelIO.C	Pass-through I/O to <code>NuclearParcel</code> , plus real read/write of the one new persisted field, <code>chi_</code> (§4)
parcels/derived/basicNuclearEulerParcel/basicNuclearEulerParcel.H	The concrete typedef + <code>is_contiguous</code> specialisation
parcels/derived/basicNuclearEulerParcel/defineBasicNuclearEulerParcel.C	<code>defineTemplateNameAndDebug</code> for the concrete parcel/cloud
parcels/derived/basicNuclearEulerParcel/makeBasicNuclearEulerParcelSubmodels.C	Instantiates the same submodel families as <code>basicNuclearParcel</code> against the new cloud type — forces (plain <i>and</i> two-phase, §6.1/§6.2), dispersion, injection, patch interaction, heat transfer, decay heat, MPPIC
parcels/include/makeNuclearEulerParcelForces.H	Registration macro for the seven two-phase force types (§5.3, §6.2)

6.3 myTurbulence library (src/myTurbulence/) — the second dispersion slot's models

File	What it is
submodels/NuclearEuler/DispersionModel/ TwoPhaseDispersionRASModel/ TwoPhaseDispersionRASModel.{H,C}	Base class for the secondary-phase dispersion models: <code>k2Field()</code> / <code>epsilon2Field()</code> read <code>NuclearEulerCloud</code> 's own persistent <code>k2()</code> / <code>epsilon2()</code> via <code>static_cast<const basicNuclearEulerCloud&></code> (§5.2)
submodels/NuclearEuler/DispersionModel/ TwoPhaseGradientDispersionRAS/ TwoPhaseGradientDispersionRAS.{H,C}	$-\nabla k_2$ -direction port of <code>GradientDispersionRAS</code> (§5.2); the only one of the three that caches an extra derived field (<code>grad(k₂)</code>), via <code>cacheFields()</code>
submodels/NuclearEuler/DispersionModel/ TwoPhaseStochasticDispersionRAS/ TwoPhaseStochasticDispersionRAS.{H,C}	Isotropic-random port of <code>StochasticDispersionRAS</code> (§5.2)
submodels/NuclearEuler/DispersionModel/ TwoPhaseVorticityDispersionRAS/ TwoPhaseVorticityDispersionRAS.{H,C}	Vorticity-biased port of <code>VorticityDispersionRAS</code> (§5.2)
parcels/include/ makeNuclearEulerParcelTurbulenceDispersionModels.H	Registration macro for the three two-phase dispersion types, into the same runtime-selection table as the primary-phase models (§5.2)
parcels/derived/basicNuclearEulerParcel/ makeBasicNuclearEulerParcelSubmodels.C	Calls both the existing primary-phase registration macro and the new one above, against <code>basicNuclearEulerCloud</code>

6.1 Why makeBasicNuclearEulerParcelSubmodels.C needed no new macros

Every `make*Models` macro used here (`makeNuclearParcelForces`, `makeParcelDispersionModels`, `makeNuclearParcelInjectionModels`, `makeParcelPatchInteractionModels`, `makeParcelStochasticCollisionModels`, `makeParcelSurfaceFilmModels`, `makeParcelNuclearHeatTransferModels`, `makeParcelDecayHeatModels`, the MPPIC macros, `makeNuclearParcelCloudFunctionObjects`) is templated purely on the `CloudType` and calls back into it only through the interface `KinematicCloud` / `NuclearCloud` already provide (`forces()`, `dispersion()`, `heatTransfer()`, `decayHeat()`, `injectors()`, ...) — every one of which `NuclearEulerCloud` inherits unchanged. Applying the identical list of macros to `basicNuclearEulerCloud` that `makeBasicNuclearParcelSubmodels.C` applies to `basicNuclearCloud` was therefore sufficient, with the file otherwise a copy of the existing one with the type name changed.

6.2 The seven two-phase force classes (src/myIntermediate/submodels/NuclearEuler/ParticleForces/)

File	What it is
------	------------

Gravity/TwoPhaseGravityForce.{H,C}	Buoyancy-corrected gravity, two phases (§5.3)
Drag/TwoPhaseSphereDrag/ TwoPhaseSphereDragForce.{H,C}	Sphere-drag correlation, two phases
Lift/TwoPhaseLiftForce/ TwoPhaseLiftForce.{H,I.H,C}	Base class: caches <code>curl(U)</code> for both phases, blends the resulting force; concrete correlations override <code>Cl()</code>
Lift/TwoPhaseSaffmanMeiLift/ TwoPhaseSaffmanMeiLiftForce.{H,C}	Saffman-Mei <code>Cl()</code> override, two phases
PressureGradient/ TwoPhasePressureGradientForce.{H,I.H,C}	DU/Dt for both phases' own real velocity fields (dict-named, default <code>U.water / U.air</code>)
VirtualMass/TwoPhaseVirtualMassForce. {H,C}	Scales the pressure-gradient result by C_{vm} ; adds the matching two-phase <code>massAdd()</code>
Thermophoretic/ TwoPhaseThermophoreticForce.{H,I.H,C}	Thermophoretic force, separate carrier thermal conductivity/mean free path per phase
BrownianMotion/ TwoPhaseBrownianMotionForce.{H,C}	Li & Ahmadi (1992) correlation, two independent per-phase random kicks, reusing <code>trackingData</code> 's cached $k1/k2$ rather than a fresh turbulence-model lookup

fvOptions coupling (src/nuclearCloudFvOptions/)

File	What it is
nuclearEulerCloudSource.H	<code>Foam::fv::nuclearEulerCloudSource</code> declaration
nuclearEulerCloudSource.C	Implementation — see §8

Test case (~/.OpenFOAM/testCases/)

Path	What it is
nuclearEulerCloudBubbleColumnTest/	Copy of the stock <code>reactingMultiphaseEulerFoam/laminar/bubbleColumn</code> tutorial, plus <code>libs / fvOptions / cloud-properties</code> additions — see §9 and §12

7. Files modified, and why

File	Change	
src/myIntermediate/Make/files	Added the two new Cloud/Parcel entries (§6)	build wiring
src/ nuclearCloudFvOptions/ Make/files	Added <code>nuclearEulerCloudSource.C</code>	build wiring
src/ nuclearCloudFvOptions/ Make/options	Added the <code>TurbulenceModels/phaseCompressible/lnInclude</code> include path (needed for <code>phaseCompressibleTurbulenceModel</code>); removed several libraries from <code>LIB_LIBS</code> — see §10	build wiring

<code>src/ nuclearCloudFvOptions/ nuclearCloudSource.C</code>	Fixed a latent gravity-lookup bug, pre-existing and never previously triggered — see §10.1	bug fix
<code>src/myTurbulence/Make/ files</code>	Added the new <code>basicNuclearEulerParcel</code> derived-parcel entry (§6.3)	build wiring

Nothing under the *existing* `submodels/` trees, no pre-existing Cloud/Parcel Template file, and no existing solver was touched. The only genuinely pre-existing file with a behavioural change is `nuclearCloudSource.C`, and that change is a bug fix, not a redesign.

7.1 Files from this feature, revised again for the dispersion rework (§5.2)

These were all created earlier for this same feature (§6), not pre-existing files — listed separately here because the dispersion rework specifically touched them a second time, on top of §6/§7's earlier additions:

File	What changed
<code>clouds/baseClasses/nuclearEulerCloud/ nuclearEulerCloud.H</code>	Added <code>vort / isRAS</code> to <code>eulerCarrierPhaseFields</code>
<code>clouds/Templates/NuclearEulerCloud/ NuclearEulerCloud.{H,I.H,C}</code>	Added <code>vort2_ / k1IsRAS_ / k2IsRAS_ / dispersion2_</code> , the <code>dispersionModel2</code> dict-key-rename construction logic, accessors, and a new <code>postEvolve()</code> override (§3, §5.2)
<code>parcels/Templates/NuclearEulerParcel/ NuclearEulerParcel.{H,C}</code>	Reinstated <code>calcDispersion()</code> , now as a χ /RAS-gated dispatcher, not a physics model (§5.2)
<code>parcels/Templates/NuclearEulerParcel/ NuclearEulerParcelTrackingDataI.H</code>	Added <code>vort2Interp_ / vort2c()</code>
<code>src/nuclearCloudFvOptions/ nuclearEulerCloudSource.{H,C}</code>	Added the persistent <code>vort2_</code> field and <code>isRASByName()</code> , passed through to the cloud constructor (§5.2, §8)

8. The fvOptions coupling layer

`nuclearEulerCloudSource` is the two-phase counterpart of the existing `nuclearCloudSource` (which feeds a plain `basicNuclearCloud` from one named phase of a `reactingMultiphaseEulerFoam` case, see the project's earlier work). It follows the identical, already-validated pattern: a runtime-selectable `fv::option`, selected in `constant/fvOptions`, that on every `fvOptions.correct(U.<phase>)` call refreshes a set of persistent carrier-field copies and calls `cloud_->evolve()`. It adds **nothing** to the phase momentum or energy matrices (`addSup / constrain` are left at the `fv::option` default, which chain down to a genuine no-op — confirmed by reading `fvOption.C`, not assumed), so it needs no new solver: any case that already runs with `reactingMultiphaseEulerFoam` can add this cloud with a `libs` entry and a dictionary.

```
// constant/fvOptions
nuclearEulerCloud1
{
    type                nuclearEulerCloudSource;
    nuclearEulerCloudSourceCoeffs
    {
        phase          water;    // primary/tracking phase
        phase2         air;      // secondary phase
        cloudName       nuclearEulerCloud1;
    }
}
```

Both named phases must exist in `constant/phaseProperties`. The two new pieces of information this class has to gather that `nuclearCloudSource` never needed are the void fractions and the turbulence closure fields:

- `phaseModel::alpha()`, like `rho()`, returns a `tmp<volScalarField>` — the same "cannot be stored by reference" trap already documented for `rho()/mu()` in this project. `alpha1c_/alpha2c_` are persistent, owned copies, refreshed every call, exactly like `rhoc_/muc_` already are.
- Per-phase turbulence is retrieved via `mesh_.lookupObject<phaseCompressibleTurbulenceModel>(IOobject::groupName(turbulenceModel::propertyName(), phaseName))` — the same lookup key ESI's own `populationBalanceModel::continuousTurbulence()` uses internally, confirmed by reading that source rather than guessed. `.k()/epsilon()` also return `tmp<>`, so `k1c_/epsilon1c_/k2c_/epsilon2c_` are likewise persistent owned copies.
- `URef()` and `thermo().T()` are, for both phases, already persistent references owned by the phase model itself (confirmed in this project's earlier work) — passed straight through with no copy, exactly as `nuclearCloudSource` already does for the primary phase.

9. Build and validation

9.1 Compilation

`wmake libso` against the live v2606 install, all three libraries, clean rebuilds, in dependency order (myIntermediate → myTurbulence → nuclearCloudFvOptions, re-running `wmake lnInclude -u` before each since new files were added at every layer):

Library	Result
<code>liblagrangianMyIntermediate.so</code>	Exit 0. (Two real issues caught and fixed during development: an unqualified <code>T_</code> reference inside a template needed <code>this->T_</code> — standard two-phase name lookup in a dependent base; and a brand-new header, <code>makeNuclearEulerParcelForces.H</code> , wasn't found on the first build after adding it — <code>wmake</code> 's flat <code>lnInclude</code> symlink farm needs an explicit <code>wmake lnInclude -u</code> to pick up newly-added files, not just changed ones.)
<code>liblagrangianMyTurbulence.so</code>	Exit 0 against the three new <code>TwoPhase*DispersionRAS</code> classes and their registration macro (§5.2, §6.3).

libnuclearCloudFvOptions.so

Exit 0, linking both of the above — the library that actually forces full instantiation of the `evolve()` / `preEvolve()` / `postEvolve()` / `calcDispersion()` call chain for a concrete `basicNuclearEulerCloud`, since `myIntermediate` alone never calls these template bodies (lazy template instantiation means a class-template member is only compiled where it's actually used).

9.2 Runtime validation

Test case: `~/OpenFOAM/testCases/nuclearEulerCloudBubbleColumnTest`, built directly on the stock, unmodified `reactingMultiphaseEulerFoam/laminar/bubbleColumn` tutorial (a real air/water bubble column, $T_{\text{water}} = 350$ K, $T_{\text{air}} = 300$ K initially — genuinely different phase temperatures, chosen deliberately so the two-phase heat-transfer closure has something non-trivial to show). Added: `libs ("libnuclearCloudFvOptions.so")`, the `fvOptions` entry from §8, and `constant/nuclearEulerCloud1Properties` (five parcels, `manualInjection`, 200 μm , `RanzMarshall` heat transfer, `NoDecayHeat`, and **all seven new two-phase forces selected at once**: `twoPhaseGravity`, `twoPhaseSphereDrag`, `twoPhaseSaffmanMeiLift`, `twoPhaseVirtualMass`, `twoPhaseThermophoretic`, `twoPhaseBrownianMotion`).

For this revision, `constant/nuclearEulerCloud1Properties` also adds `dispersionModel2 twoPhaseGradientDispersionRAS`; (§5.2) alongside the unchanged, primary-phase `dispersionModel none`; . Ran to completion on the unmodified `reactingMultiphaseEulerFoam` binary (0.1 s, ΔT 0.005):

```
Selecting dispersion model none
Selecting dispersion model twoPhaseGradientDispersionRAS

Constructing particle forces
  Selecting particle force twoPhaseGravity
  Selecting particle force twoPhaseSphereDrag
  Selecting particle force twoPhaseSaffmanMeiLift
  Selecting particle force twoPhaseVirtualMass
  Selecting particle force twoPhaseThermophoretic
  Selecting particle force twoPhaseBrownianMotion

Solving2-D cloud nuclearEulerCloud1
Cloud: nuclearEulerCloud1
  Current number of parcels      = 5
  ...
  Temperature min/max           = 350, 350
  Secondary-phase volume fraction min/max = 2.62857e-05, 2.62857e-05
  Secondary-phase temperature min/max    = 349.943, 350

ExecutionTime = 0.86 s  ClockTime = 0 s
...
End
```

This confirms the $\chi=0 / w_2 \rightarrow 0$ consistency property from §5.1/§5.3 across every one of the seven forces plus heat transfer at once, not just "it ran". All seven new two-phase forces were constructed and evaluated every sub-step (visible in the "Selecting particle force" log lines), each internally computing both an F_1 and an F_2 contribution and blending them with $\chi=0$ - and the parcels' behaviour was indistinguishable from a plain, single-phase run: temperature converged to exactly 350 K (the pure-water value), and the run reproduced the same

trajectory/diagnostics as the §9.2 validation before the force framework existed. The `NuclearEulerCloud::info()` diagnostic lines shown above (`Secondary-phase volume fraction/temperature min/max`) are themselves new, added purely for this kind of at-a-glance check.

The dispersion rework (§5.2) is confirmed the same way. The "Selecting dispersion model `twoPhaseGradientDispersionRAS`" log line proves `NuclearEulerCloud`'s constructor correctly renamed `dispersionModel2` to `dispersionModel` in its throwaway dict copy and that the model was found in the shared runtime-selection table (§5.2) — and the run's numbers are, correctly, *identical* to the pre-dispersion-rework baseline, since both phases are laminar in this tutorial (`k2IsRAS()` is false) and χ is always 0: the new dispatch branch in `calcDispersion()` is proven reachable-by-selection but is not yet exercised at the `update()` call itself, exactly as the design intends until a RAS-turbulent secondary phase and a χ -switching model both exist (§11).

10. A real bug found along the way (and one ruled out)

10.1 Fixed: gravity lookup in nuclearCloudSource (pre-existing, previously untested)

The very first attempt at the 0.1 s validation run failed immediately with:

```
--> FOAM FATAL ERROR:
      failed lookup of g (objectRegistry region0)
```

Both the new `nuclearEulerCloudSource` and the *existing*, previously-compiled-but-never-runtime-tested `nuclearCloudSource` looked gravity up as `mesh_.lookupObject<uniformDimensionedVectorField>("g")`. This project had already discovered (validating the single-phase `nuclearCloudSourceSingle` against `buoyantBoussinesqPimpleFoam`) that this version of OpenFOAM does *not* register `g` as a plain field — it's a `meshObjects::gravity` `MeshObject` cached on the Time database instead — and had already fixed it in `nuclearCloudSourceSingle.C`. `nuclearCloudSource.C` itself (the phase-based variant) had the identical bug, simply never triggered before because it had only ever been compiled and linked, never run against a real multiphase case (confirmed by this project's own prior notes). Fixed identically in both files:

```
#include "gravityMeshObject.H"
...
meshObjects::gravity::New(mesh_.time()) // was: mesh_.lookupObject<uniformDimensionedVectorField>("g")
```

10.2 Investigated and ruled out: an unrelated solver/fvOptions interaction

Extending the same case to 2 s (to let the bubble plume actually reach the tracked particles and produce a more dramatic demonstration of §5.1) diverged partway through, crashing with a SIGFPE inside the base solver's own `SchillerNaumann::CdRe()` drag correlation. Given this project's prior experience with a superficially similar issue in `src/flotation` (a redundant explicit library link into a dynamically-loaded

fvOptions plugin corrupting runtime-selection-table state), that was the first suspect, and was investigated thoroughly:

1. Disabled `cloud_ -> evolve()` (particle tracking never runs) — still diverged, identically.
2. Disabled `updateCarrierFields()` entirely, i.e. `correct()` became a complete no-op — still diverged, identically, down to the exact same corrupted numbers.
3. Removed every library from `nuclearCloudFvOptions` 's `LIB_LIBS` that overlaps with what `reactingMultiphaseEulerFoam` already links (`reactingMultiphaseSystem` , the turbulence-model libraries, the thermophysical libraries), and reworked the one remaining genuinely necessary lookup (`phaseSystem` , needed for its RTTI) to go through the always-linked, core `IOdictionary` base and a `static_cast` instead — eliminating the need to link `reactingMultiphaseSystem` at all. This is good practice independent of this specific bug (kept), but the divergence persisted, with identical numbers, even after removing the link entirely.
4. **Decisive test:** built a from-scratch, completely unrelated, minimal `fv::option` (`trivialSource` , in a throwaway `src/rstTest` library touching nothing from `phaseSystem` , `lagrangian` , or `myIntermediate` — just `fvOption.H` and an empty `correct()`), selected it on `U.water` in an otherwise-vanilla copy of the same tutorial, and reran to 2 s. **It diverged identically**, same crash location, same corrupted field values.

Conclusion: this is a pre-existing instability in `reactingMultiphaseEulerFoam` 's interaction with *any* custom `fv::option` selected against a phase momentum field in this specific tutorial/configuration — **not a defect in `NuclearEulerCloud` / `NuclearEulerParcel` , not a defect in `nuclearEulerCloudSource` , and not a linking issue.** The vanilla, completely unmodified tutorial (no `libs` , no `fvOptions` at all) runs cleanly to 2 s with no changes. The throwaway `trivialSource` library and its test case were deleted after establishing this; the finding is recorded here so it isn't rediscovered from scratch. It's out of scope to chase further here, but worth logging with OpenFOAM/reporting upstream if this cloud is used for longer production runs on this solver family.

10.3 Discovered: `particleForces` has a different dict convention than the single-named-model selectors

Wiring up coeffs for the new lift/virtual-mass/thermophoretic/Brownian forces initially failed with:

```
--> FOAM FATAL IO ERROR:
Force twoPhaseSaffmanMeiLift must be specified as a dictionary
```

The instinct, by analogy with `heatTransferModel RanzMarshall; + a sibling RanzMarshallCoeffs{} dict` at the same `subModels` level, was to write `twoPhaseSaffmanMeiLift;` (bare) plus a sibling `twoPhaseSaffmanMeiLiftCoeffs{} .` That convention is specific to single-named-model selectors (`HeatTransferModel::New(dict, owner)` , where `dict` is the whole `subModels` dict and the coeffs are read from a `<type>Coeffs` sibling inside it). `particleForces` is a *list*, handled by a different class, `ParticleForceList` , which iterates the entries of the `particleForces{} dict` itself: a bare entry (`twoPhaseGravity;`) passes the whole `particleForces{} dict` through to `ParticleForce::New()` , used only when the force needs no coeffs at all (`readCoeffs=false` , e.g. plain `gravity` / `sphereDrag` and their two-phase counterparts); a force that *does* need coeffs must instead be written as its own dictionary entry,

named after the force, holding those coeffs directly - confirmed by reading `ParticleForce.C`'s constructor, which literally checks `coeffs_.dictName() == forceType`. Fixed by nesting each coeffs-requiring force as its own named sub-dict directly inside `particleForces{}` (see §12 for the corrected form) - a one-line-per-force fix once the actual mechanism was understood, but worth recording since it isn't obvious from the single-named-model examples elsewhere in this same dictionary.

11. Known limitations and natural next steps

- **χ has a placeholder test signal, not a physical switching model (§13.2).** A dictionary-tunable `chi(t) = 1 - exp(-gamma*age)` now exists purely to exercise the χ -dependent machinery end-to-end; something still needs to decide, physically, when a tracked particle should be treated as belonging to the secondary phase (e.g. attached to a bubble) and set `chi()` accordingly. The force framework (§5.3) needs no changes the moment such a model exists - it already consumes a non-zero, time-varying χ correctly (§13.3).
- **Only one drag and one lift correlation got two-phase siblings (§5.4)** — `sphereDrag` / `SaffmanMeiLift` only, not the fork's other variants (`WenYuDrag`, `ErgunWenYuDrag`, `PlessisMasliyahDrag`, `NonSphereDrag`, `TomiyamaLift`). Each would follow the identical pattern (evaluate twice, blend by χ) and is a natural, low-risk follow-up.
- **Two-phase turbulent dispersion (§5.2) is built and wired end-to-end, but still gated shut by `k2IsRAS()` in every validated case so far.** Both dispersion RAS models that remain (§13.1 - the vorticity-based one was removed as unhelpful) are proven to select/construct correctly and, since the §13.3 crash fix, to cache safely when RAS-gated off - but the actual `dispersion2().update()` call has still never fired, since the validation tutorial keeps both phases laminar. Meaningfully validating it needs a case with a genuinely RAS-turbulent secondary phase (e.g. `turbulenceProperties.air` set to `kEpsilon`, with the accompanying `k.air/epsilon.air/nut.air/alphat.air` field files the stock laminar tutorial doesn't ship) — deliberately not attempted here, since building out that turbulence setup is orthogonal to the mechanism itself and would risk being confounded with the unrelated §10.2 instability.
- **The cloud's domain-wide "Secondary-phase volume fraction min/max" `info()` diagnostic is broken (§13.3):** it prints a fixed value regardless of the real, verified-varying `alpha.air` field. Pre-existing (confirmed present in earlier, differently-configured runs too), reporting-only, does not affect any physics - the particle-local values the forces/heat-transfer actually use are confirmed working via the parcel temperature response (§13.3). Worth a focused fix, not chased down here.
- **Only two of the original three `myTurbulence` RAS dispersion correlations were ported, and one of those two was since removed.** `FunctionYPlusDispersionRAS` (wall-distance/y+-dependent) was never ported; `TwoPhaseVorticityDispersionRAS` was ported, tested, and removed as not useful (§13.1). Only `TwoPhaseGradientDispersionRAS` / `TwoPhaseStochasticDispersionRAS` remain.
- **One-way coupling only.** Like the existing single-phase `fvOptions` integration this mirrors, the cloud reads the flow and never feeds back into either phase's equations. The `UTrans` / `hsTrans` / ... source-term bookkeeping inherited unchanged from `NuclearCloud` is still accumulated (against the primary phase), ready for a future `addSup(alpha, rho, eqn, fieldi)` override if two-way coupling is ever wanted — no architecture change needed, per this project's own earlier notes on the single-phase case.

- **The §10.2 solver instability** is worth a focused investigation in its own right if this cloud is to be used for production-length runs on `reactingMultiphaseEulerFoam`, independent of anything built here.

12. How to use it in a case

1. Build both libraries: `wmake libso $WM_PROJECT_USER_DIR/src/myIntermediate` then `wmake libso $WM_PROJECT_USER_DIR/src/nuclearCloudFvOptions`.
2. In `system/controlDict`: `libs ("libnuclearCloudFvOptions.so");`
3. In `constant/fvOptions`: a `nuclearEulerCloudSource` entry naming the primary (`phase`) and secondary (`phase2`) phases, as in §8. Both must exist in `constant/phaseProperties`.
4. A `constant/<cloudName>Properties` dictionary, in exactly the same format any `NuclearCloud`-based case already uses (`constantProperties`, `subModels`, `solution`) — see `nuclearEulerCloudBubbleColumnTest/constant/nuclearEulerCloud1Properties` for a complete worked example, including the one new optional entry, `alphaSmall` (default 10^{-6} , §5.1).
5. `solution.interpolationSchemes` needs an entry for every carrier field name: the primary phase's real solver fields keep their own names (e.g. `U.water`, `T.water`); everything else is a persistent copy named `<fvOptionName>:<field>` (e.g. `nuclearEulerCloud1:alpha2`) — a single wildcard entry (`"*" cell;`) covers all of them without enumerating each one by hand.
6. To select the new two-phase forces (§5.3), remember the §10.3 dict convention: a bare `name;` for a force needing no coeffs, or a dictionary named after the force holding its coeffs directly - *not* a separate `<name>Coeffs` sibling:

```
subModels
{
    particleForces
    {
        twoPhaseGravity;
        twoPhaseSphereDrag;

        twoPhaseVirtualMass
        {
            Cvm 0.5;
        }

        twoPhaseThermophoretic
        {
            Kp 1.0; Kc1 0.6; Kc2 0.026;
            lambda1 1e-9; lambda2 6.8e-8;
        }
    }
    ...
}
```

7. To give the secondary phase its own turbulent dispersion (§5.2), add a `dispersionModel2` entry alongside the existing `dispersionModel` — it is only ever consulted for a parcel with $\chi > 0$ whose secondary phase is RAS-turbulent, so it is safe to add speculatively (e.g. before a χ -switching model exists) with no effect on current behaviour:

```
subModels
{
    dispersionModel      none;                // primary phase, unchanged
    dispersionModel2     twoPhaseGradientDispersionRAS; // or twoPhaseStochasticDispersionRAS
    ...
}
```

13. Revision: removing vorticity dispersion, a placeholder χ -switch, and a real two-phase test

Three follow-up changes, made after the field validation of §9 confirmed the dispersion2 mechanism selects and constructs correctly but was never numerically exercised (χ always 0, no RAS-turbulent phase in the tutorial).

13.1 Vorticity dispersion removed

`TwoPhaseVorticityDispersionRAS` was tested and found not useful, and was removed entirely on instruction - the whole class, its registration, and every field that existed purely to feed it: `eulerCarrierPhaseFields::vort`, `NuclearEulerCloud::vort2_ / vort2()`, `NuclearEulerParcel::trackingData::vort2Interp_ / vort2c()`, and `nuclearEulerCloudSource`'s persistent `vort2_` copy. `TwoPhaseGradientDispersionRAS` and `TwoPhaseStochasticDispersionRAS` are unaffected and remain selectable; `calcDispersion()`'s dispatcher now passes a dummy `vector::zero` for `dispersion2().update()`'s unused vorticity argument (a fixed part of the shared `DispersionModel::update()` signature that neither remaining model reads).

13.2 A placeholder χ -switching model

χ had no model setting it to anything but 0 (§5.3/§11). A new, explicitly-placeholder model was added purely to exercise the χ -dependent machinery: a new `constantProperties` entry, `chiSwitchGamma` (default 0, so an existing case with no explicit entry keeps $\chi=0$ - today's behaviour, unchanged), and `NuclearEulerParcel::updateChi()`, called once per `calc()` ahead of anything that reads `chi()` (the two-phase forces, via `calcVelocity()`):

$$\chi(t) = 1 - e^{-\gamma \cdot \text{age}}$$

This is deliberately *not* a physical switching model (e.g. detecting particle–bubble attachment, still future work) - just a smoothly time-varying, dictionary-tunable test signal that takes every χ -gated code path from dormant to genuinely exercised.

13.3 A real two-phase test case, and a crash it found

Two changes to `nuclearEulerCloudBubbleColumnTest` make the switch actually mean something: `chiSwitchGamma 20`; added to `nuclearEulerCloud1Properties` ($\chi \approx 0.33/0.63/0.86$ at $t = 0.02/0.05/0.1$ s), and a second `setFieldsDict` region overriding `alpha.air 0.3 / alpha.water 0.7` in a box bracketing every `nuclearEulerCloud1Positions` injection point (previously that whole region was pure water, `alpha.air 0`) - without it, the local secondary-phase volume fraction the particles actually see would stay zero regardless of χ , and the two-phase machinery would still have nothing real to blend.

Rerunning with both changes crashed immediately with a SIGSEGV, cleanly localised by the stack trace to `fvc::grad` inside `TwoPhaseGradientDispersionRAS::cacheFields()`. The cause:

`NuclearEulerCloud::preEvolve() / postEvolve()` called `dispersion2_>cacheFields(true/false)` *unconditionally* every `evolve()`, computing `grad(k2())` even when the secondary phase is laminar (as in this tutorial) - and a laminar phase's `k()` is whatever placeholder value its turbulence model happens to return, not something `fvc::grad`'s boundary handling can safely be assumed to operate on. **Fixed** by gating both calls on `k2IsRAS_`, exactly mirroring the gate `calcDispersion()` already applies to the `update()` call itself - there was never a reason to cache anything for a non-RAS secondary phase in the first place, since `dispersion2` is physically meaningless there regardless of χ .

With the fix in place, the case runs clean to completion (`EXIT=0`) with the void-fraction pocket and χ rising from 0. The result is visibly different from every earlier $\chi=0$ run: parcel temperature, previously locked at exactly 350 K throughout, now dips to ~343–349 K early in the run before recovering as the particles move - direct evidence the local secondary-phase fields are now actually influencing the tracked particles, not just being read and discarded. (The cloud's own `info()` diagnostic line, "Secondary-phase volume fraction min/max", still prints a fixed value regardless of the real domain field - confirmed, by directly parsing the written `alpha.air` field, to be a pre-existing reporting-only quirk unrelated to the physics and predating this revision; logged as a loose end in §11.)

13.4 Updated limitation

χ is no longer *always* 0 - a real, if deliberately non-physical, time-varying signal now exists and is wired end-to-end through every χ -gated code path. The genuinely missing piece (§11) is narrower than before: a physically-motivated switching criterion (e.g. bubble attachment) to replace `chiSwitchGamma`'s placeholder exponential.

14. Revision: χ as a boolean stochastic switch, vorticity removed fork-wide, FunctionYPlusDispersionRAS retired

14.1 χ is now a real `bool`, and switches stochastically per Eulerian step

Two follow-up corrections to §13.2's placeholder: χ must be a genuine discrete state - "the particle is either in the salt (liquid) or in the gas" - not a [0,1] scalar that happens to only ever take the values 0 or 1, and the switching test itself should be a proper per-timestep Bernoulli trial rather than a cumulative-probability threshold crossing.

`NuclearEulerParcel::chi_` is now declared `bool` (previously `scalar`), with matching `bool chi()` `const / bool& chi()` accessors. This needed **zero changes** to any of the seven two-phase force classes (§5.3): each already computes `const scalar chi = p.chi();`, and C++ converts `bool` $\rightarrow$ `scalar` implicitly (0.0/1.0), so $\mathbf{F} = \mathbf{F}_1(1-\chi) + \mathbf{F}_2\chi$ is unaffected. I-O switched from `IOfield<scalar>` to `IOfield<bool>` (a first-class OpenFOAM alias, `boolIOfield.H`) and the matching `cloud::lookupIOfield<bool>/createIOfield<bool>` calls for the objectRegistry read/write path.

`NuclearEulerParcel::updateChi()` (still called once per `calc()`, still explicitly a test placeholder, not a physical model) now works like this:

$$p_{\text{switch}} = 1 - e^{-\gamma \Delta t}, \quad \text{switch if } U(0,1) < p_{\text{switch}}$$

- **One-way only:** once $\chi=\text{true}$ (secondary phase), an early-return guard at the top of `updateChi()` makes it stick - it never reverts to false, matching the "either/or, not back-and-forth" semantics.
- **Evaluated against the Eulerian time step, not the Lagrangian tracking sub-step.** `calc()`'s own `dt` argument can be smaller than the flow solver's actual time step whenever a parcel crosses a mesh face partway through it (`KinematicCloud`'s face-crossing sub-cycling). Since the requirement was explicitly "at each Eulerian step (time step of the simulation)", Δt is read as `cloud.mesh().time().deltaTValue()` - the real solver `deltaT` - not the possibly-smaller per-substep `dt` that happened to be passed into this particular `calc()` call.
- Draws `cloud.rndGen().template sample01<scalar>()` - the same uniform-[0,1) generator the existing `Stochastic...DispersionRAS` models already use for their own random kicks (§5.2), so no new RNG machinery was introduced. Needs the `.template` disambiguator ahead of `sample01<scalar>` because `cloud` is itself a template parameter (`TrackCloudType`) here, making the whole expression dependent - the compiler can't otherwise tell whether `<` opens a template-argument list or means "less than" until `TrackCloudType` is actually instantiated.
- `gamma` is still `constantProperties::chiSwitchGamma` (default 0 $\rightarrow \chi$ stays false forever unless a case opts in - unchanged, existing cases still unaffected).

14.2 Vorticity removed from the entire fork, not just the two-phase additions

§13.1 removed the two-phase vorticity dispersion model and the fields built solely to feed it. This revision goes further, on explicit instruction, and removes the carrier vorticity concept from the *whole* myIntermediate/myTurbulence template stack - because the one thing that ever consumed it, vorticity-based dispersion, is retired for good (§14.3 removes the last surviving vorticity-adjacent model too, for a different reason).

Layer	What was removed
<code>DispersionModel::update()</code> (<code>myIntermediate</code> , <code>base</code>)	The <code>const vector& vortc</code> parameter dropped from the shared pure-virtual signature - a breaking change to every implementer
<code>NoDispersion</code> , <code>DispersionRASModel</code> , <code>GradientDispersionRAS</code> , <code>StochasticDispersionRAS</code> , <code>FunctionYPlusDispersionRAS</code> (at the time), both <code>TwoPhase*</code> models	Matching <code>update()</code> override signatures updated. <code>NoDispersion.C</code> 's out-of-line definition had the parameter <code>unnamed</code> - easy to miss with a plain text search for <code>vortc</code> , since only the type appeared, not the name
<code>KinematicCloud</code> , <code>KinematicParcel::trackingData</code>	<code>vort_ / vort()</code> and <code>vortInterp_ / vortc()</code> removed - the carrier vorticity field and its per-parcel interpolator no longer exist at all
<code>NuclearCloud</code> , <code>ThermoCloud</code> , <code>NuclearEulerCloud</code>	Each had its own duplicate <code>vort_</code> reference (passed through from the <code>KinematicCloud</code> constructor call) and <code>vort()</code> accessor - all removed, and the constructor signatures shortened to match
<code>kinematicCloudSource</code> , <code>nuclearCloudSource</code> , <code>nuclearCloudSourceSingle</code> , <code>nuclearEulerCloudSource</code>	The persistent <code>vort_</code> field and its <code>fvc::curl(U)</code> refresh removed from all four <code>fvOptions</code> - nothing downstream reads it any more
<code>msfrParcelFoam</code> , <code>nuclearParcelFoam</code> (solvers)	The only two solvers that ever constructed a vorticity field for the cloud (<code>createFields.H</code>) and refreshed it every PIMPLE iteration (<code>vort = fvc::curl(U);</code> in the main <code>.C</code>) - both the field and the refresh line removed, and <code>createClouds.H</code> 's cloud-constructor call shortened to match

All three libraries and both solvers were rebuilt (clean) and the two-phase test case rerun end-to-end (`EXIT=0`, identical force/dispersion selection and numerics to the pre-removal baseline) to confirm nothing broke.

14.3 FunctionYPlusDispersionRAS retired

The last of the four original single-phase RAS dispersion correlations never ported to a two-phase sibling (§5.4/§11) - wall-distance/ y^+ -dependent, and explicitly "won't be used again" - was deleted outright: its directory removed, and its `#include / makeDispersionModelType` registration dropped from `makeParcelTurbulenceDispersionModels.H`. Only `GradientDispersionRAS` and `StochasticDispersionRAS` remain as primary-phase RAS dispersion options in this fork.

Deleting the header without a full dependency-graph refresh broke the very next `myTurbulence` build:
`wmakeLnInclude -u .` correctly dropped the dead symlink, but a *stale* `Make/<platform>/...C.dep` file from an earlier build still listed `FunctionYPlusDispersionRAS.H` as a dependency, and `wmake`'s incremental logic trusts that stale list rather than re-scanning includes from scratch - No rule to make target `'lnInclude/FunctionYPlusDispersionRAS.H'`. Fixed with `rm -rf Make/<platform>` to force a full dependency-graph rebuild. A different trigger from the earlier UNC-bridge edit-staleness gotcha (§9.1) - that one is about edits silently not taking effect; this one is specifically about *deleting* a file a `.dep` still names.

15. Revision: TwoPhaseBrownianMotionForce moved to myTurbulence

The other six two-phase forces (§5.3) are two-phase siblings of single-phase forces that live in `myIntermediate`, so keeping their two-phase versions there too was consistent. `BrownianMotionForce` is the exception: the single-phase original lives in `myTurbulence` (`submodels/Thermodynamic/ParticleForces/BrownianMotion/`), not `myIntermediate` - so `TwoPhaseBrownianMotionForce` was moved there too, into a new `submodels/NuclearEuler/ParticleForces/BrownianMotion/` directory, to sit next to what it mirrors.

15.1 Registration: a new macro, populating the same shared table

Removed from `myIntermediate`'s `makeNuclearEulerParcelForces.H` (down to six forces); added to a new `makeNuclearEulerParcelTurbulenceForces.H` in `myTurbulence`, mirroring `makeThermoParcelTurbulenceForces.H`'s existing role for the single-phase force. Called from `makeBasicNuclearEulerParcelSubmodels.C` in `myTurbulence`, it adds `TwoPhaseBrownianMotionForce` into the exact same `ParticleForce<kinematicCloudType>` runtime-selection table `myIntermediate`'s macro populates for the other six - cross-library population of one shared table is already the established pattern this project uses for the two dispersion-model tables (§5.2), so this needed no new plumbing, only another call site.

15.2 Rewritten to be a genuine two-phase port, not a trackingData shortcut

The original implementation (§5.3) took a shortcut: rather than looking up a turbulence model in the mesh registry the way the single-phase `BrownianMotionForce::kModel()` does, it read `td.k1c()/td.k2c()` - the primary/secondary phase turbulence kinetic energy `NuclearEulerParcel::trackingData` already interpolates for other purposes. That was flagged as a deliberate simplification at the time (§5.3's implementation notes). This revision removes it, porting the original's actual mechanism instead:

- `kModel()` (primary phase) is now *identical* to the original: locates the registered `turbulenceModel` for `this->owner().U().group()` via `obr.findObject<turbulenceModel>(...)` and returns its `k()`. No cast needed - every symbol it touches (`owner().mesh()`, `owner().U()`) already exists on the fixed `kinematicCloudType` layer.
- A new `k2Model()` mirrors it for the secondary phase: `static_cast<const basicNuclearEulerCloud&>(this->owner()).U2().group()` supplies the phase-2 group name (the same `static_cast`-for-`owner()` pattern the two-phase dispersion models already use, §5.2), and the rest of the lookup is byte-for-byte the same as `kModel()`.
- `cacheFields()`/the destructor now do the full `tmp<>/ownership` dance *twice* (`kPtr_ / ownK_` and `k2Ptr_ / ownK2_`), exactly mirroring the original's single-field version.
- Picked up the original's `spherical` dict option (`useSpherical_`, default true) and its `rndGen_` member for full structural parity - including the original's own minor quirk of storing `rndGen_` in the

constructor but re-fetching `this->owner().rndGen()` locally in `calcCoupled()` anyway;
reproduced faithfully rather than "fixed", since the ask was fidelity to the original, not improving on it.

All three libraries plus `nuclearCloudFvOptions` rebuild clean after the move, and the test case reruns end-to-end (`EXIT=0`) with `twoPhaseBrownianMotion` now correctly reported as coming from `myTurbulence` in the selection log. The test case sets `turbulence no;` for this force, so the new `kModel()` / `k2Model()` registry lookups are proven to compile, link and select correctly, but - like §14's `dispersion2 update()` - have not yet been exercised at runtime; that needs a case with `turbulence yes;` and real registered turbulence models on both phases.

tom-v2606 · `src/myIntermediate`, `src/myTurbulence`, `src/nuclearCloudFvOptions`, `src/kinematicCloudFvOptions`, `applications/solvers/msfrParcelFoam`, `applications/solvers/nuclearParcelFoam` · validated against OpenFOAM v2606